Sistema de ingesta continua con Kafka y backups diarios con Salt Minion

Raúl Cristian Dumitru Tanase
2º CFGM SMR
01/03/2026

ÍNDICE - ANEXO 1

1. Preámbulo **2**
2. Archivo /compose/compose.yaml **2**
- Servicio zookeeper 2
- Servicio kafka 2
- Servicio Postgres 3
- Servicio Adminer 4
- Servicio Salt Minion 5

3. Archivo /compose/.env **5**
4. Archivo /db/init.sql **6**
5. Archivo /salt/backup_db.sh **7**
6. Archivo /salt/dockerfile **8**

1. Preámbulo

- En este anexo vamos a estar descomponiendo los componentes que conforman la carpeta /compose y /db para entender el por qué de cada línea de código y en qué afecta dentro del proyecto.

2. Archivo /compose/compose.yaml

- ***Servicio zookeeper***

```
1  zookeeper:
2    image: confluentinc/cp-zookeeper:7.6.1
3    container_name: SOR-zookeeper
4    environment:
5      ZOOKEEPER_CLIENT_PORT: 2181
6      ZOOKEEPER_TICK_TIME: 2000
7    ports:
8      - "2181:2181"
```

- **Línea 1:** Definir el servicio, en este caso zookeeper.
- **Línea 2:** Elegimos la imagen y versión de zookeeper a usar.
- **Línea 3:** Nombre del contenedor, fácil identificación.
- **Línea 4:** Definir el apartado de variables de entorno.
- **Línea 5:** Puerto desde el cual escucha zookeeper al cliente.
- **Línea 6:** Timing interno para zookeeper.
- **Línea 7:** Definir apartado de puertos del servicio.
- **Línea 8:** Mapeo de puertos donde interpretamos “HOST:CONTAINER”

- ***Servicio kafka***

```
kafka:
  image: confluentinc/cp-kafka:7.6.1
  container_name: SOR-kafka
  depends_on:
    - zookeeper
  ports:
    - "9094:9094"
  environment:
    KAFKA_BROKER_ID: 1
```

```
10      KAFKA_ZOOKEEPER_CONECT: "zookeeper:2181"
11                                                  KAFKA_LISTENERS:
   "PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9094"
12                                          KAFKA_ADVERTISED_LISTENERS:
   "PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9094"
13                              KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
   "PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT"
14      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
15      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
16      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
```

- **Línea 1:** Definir el servicio, en este caso kafka.
- **Línea 2:** Elegimos la imagen y versión de kafka a usar.
- **Línea 3:** Nombre del contenedor, fácil identificación.
- **Línea 4:** Definir apartado de dependencia.
- **Línea 5:** Definimos que depende de zookeeper.
- **Línea 6:** Definir apartado de puertos del servicio.
- **Línea 7:** Mapeo de puertos donde interpretamos “HOST:CONTAINER”
- **Línea 8:** Definir el apartado de variables de entorno.
- **Línea 9:** ID del broker.
- **Línea 10:** Define la forma de conexión al tener dependencia activa.
- **Línea 11:** Abrimos puertas para aceptar conexiones - 9092 = Docker y 9094: Windows host local.
- **Línea 12:** Indica a los host anteriores el cómo conectarse al servicio.
- **Línea 13:** Protocolo de seguridad para los listeners.
- **Línea 14:** Protocolo de seguridad para los brokers (debería dar igual al haber solo 1).
- **Línea 15:** Factor para los offset de kafka (default = 3, nuestro lab = 1).
- **Línea 16:** Crea topics cuando un producer escribe para uno que no existe.

- ***Servicio Postgres***

```
1  postgres:
2    image: postgres:16-alpine
3    container_name: SOR-postgres
4    environment:
5      POSTGRES_DB: ${POSTGRES_DB}
6      POSTGRES_USER: ${POSTGRES_USER}
7      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
8    ports:
```

```
9       - "5432:5432"
10    volumes:
11      - SOR_pgdata:/var/lib/postgresql/data
12      - ../db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

- **Línea 1:** Definir el servicio, en este caso postgres.
- **Línea 2:** Elegimos la imagen y versión de postgres a usar.
- **Línea 3:** Nombre del contenedor, fácil identificación.
- **Línea 4:** Definir el apartado de variables de entorno.
- **Línea 5:** Nombre de la base de datos, sustitución desde el archivo .env
- **Línea 6:** Nombre de usuario, sustitución desde el archivo .env
- **Línea 7:** Contraseña, sustitución desde el archivo .env
- **Línea 8:** Definir apartado de puertos del servicio.
- **Línea 9:** Mapeo de puertos donde interpretamos “HOST:CONTAINER”
- **Línea 10:** Definir apartado de volúmenes.
- **Línea 11:** Volúmen para guardar información de la DB fuera del container.
- **Línea 12:** Volúmen para guardar la configuración de la DB fuera del container.

- ***Servicio Adminer***

```
1  adminer:
2    image: adminer:latest
3    container_name: SOR-adminer
4    depends_on:
5      - postgres
6    ports:
7      - "8081:8080"

8 volumes:
9  SOR_pgdata:
```

- **Línea 1:** Definir el servicio, en este caso adminer.
- **Línea 2:** Elegimos la imagen y versión de adminer a usar.
- **Línea 3:** Nombre del contenedor, fácil identificación.
- **Línea 4:** Definir apartado de dependencia.
- **Línea 5:** Definimos que depende de postgres.
- **Línea 6:** Definir apartado de puertos del servicio.
- **Línea 7:** Mapeo de puertos donde interpretamos “HOST:CONTAINER”
- **Línea 8:** Definir apartado de volúmenes para todo el compose.
- **Línea 9:** Volúmen para guardar bien todo el compose.

- ***Servicio Salt Minion***

```
1 salt-minion:
2   build:
3     context: ../salt
4   container_name: SOR-salt-minion
5   depends_on:
6     - postgres
7   environment:
8     PGHOST: postgres
9     PGPORT: 5432
10    PGDATABASE: ${POSTGRES_DB}
11    PGUSER: ${POSTGRES_USER}
12    PGPASSWORD: ${POSTGRES_PASSWORD}
13  volumes:
14    - ../backups:/backups
15    - ../salt:/srv/salt:ro
  restart: unless-stopped
```

- **Línea 1:** Definir el servicio, en este caso salt-minion.
- **Línea 2:** Definir apartado de build.
- **Línea 3:** Damos contexto de donde se encuentra la build de la imagen.
- **Línea 4:** Nombre del contenedor, fácil identificación.
- **Línea 5:** Definimos apartado de dependencia.
- **Línea 6:** Definimos que depende de postgres.
- **Línea 7:** Definir el apartado de variables de entorno.
- **Línea 8:** Definimos el host de la base de datos.
- **Línea 9:** Definimos el puerto de comunicación con postgres.
- **Línea 10:** Nombre de la base de datos, sustitución desde el archivo .env
- **Línea 11:** Nombre de usuario, sustitución desde el archivo .env
- **Línea 12:** Contraseña, sustitución desde el archivo .env
- **Línea 13:** Definir apartado de volúmenes.
- **Línea 14:** Definimos la relacion entre la máquina local y el container.
- **Línea 15:** Definimos la relación entre la carpeta de scripts local y el container.

3. Archivo /compose/.env

```
1 # Postgres
2 POSTGRES_DB=SOR_db
3 POSTGRES_USER=SOR_dev
4 POSTGRES_PASSWORD=Los-Boscos-SOR-5439

5 # Kafka
6 KAFKA_TOPIC=SOR_sensores
```

- **Línea 1:** Comentario para saber que hablamos de postgres.
- **Línea 2:** Definimos el nombre de la base de datos.
- **Línea 3:** Definimos el nombre de usuario para la DB.
- **Línea 4:** Definimos la password del usuario de la DB.
- **Línea 5:** Comentario para saber que hablamos de kafka.
- **Línea 6:** Valor para crear posteriormente el topic de kafka (no lo crea automaticamente).

4. Archivo /db/init.sql

```
1 CREATE TABLE IF NOT EXISTS measurements (
2   id BIGSERIAL PRIMARY KEY,
3   device_id TEXT NOT NULL,
4   ts TIMESTAMPTZ NOT NULL,
5   temp DOUBLE PRECISION,
6   hum DOUBLE PRECISION,
7   raw JSONB
);

8 CREATE INDEX IF NOT EXISTS idx_measurements_ts ON measurements (ts
DESC);
```

- **Línea 1:** Crea la tabla measurements si no existiera ya.
- **Línea 2:** Básicamente es la llave primaria que genera un identificador único por línea.
- **Línea 3:** Identificador del componente arduino - De texto - Obligatorio.
- **Línea 4:** Momento de la medición - De tiempo - Obligatorio.
- **Línea 5:** Temperatura - FloatXX - No obligatorio.
- **Línea 6:** Humedad - FloatXX - No obligatorio.

- **Línea 7:** Conserva el mensaje original de kafka aunque luego haya cambios de interfaz.
- **Línea 8:** Crea un índice donde se vean los datos por tiempo descendente (más lento pero más ordenado).

5. Archivo /salt/backup_db.sh

```
1 #!/bin/bash
2 set -e

3 TS=$(date +"%Y-%m-%d_%H%M%S")
4 OUT="/backups/backup_${PGDATABASE}_${TS}.sql"

5 echo "[INFO] Creating backup: $OUT"
6 pg_dump -h "$PGHOST" -p "$PGPORT" -U "$PGUSER" -d "$PGDATABASE" >
"$OUT"
7 echo "[OK] Backup created: $OUT"
8 ls -lh "$OUT"
```

- **Línea 1:** Ejecuta el archivo con bash.
- **Línea 2:** Define que si algun comando falla, el script para.
- **Línea 3:** Es una variable que define el nombre del archivo.
- **Línea 4:** Es una variable que define la ruta del archivo.
- **Línea 5:** Mensaje informativo.
- **Línea 6:** Creación de la copia de seguridad.
- **Línea 7:** Mensaje informativo.
- **Línea 8:** Lista el archivo para confirmar que efectivamente se ha creado.

6. Archivo /salt/dockerfile

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
 && apt-get install -y ca-certificates curl gnupg lsb-release tzdata
salt-minion \
 && mkdir -p /etc/apt/keyrings \
```

```
 && curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc \
    | gpg --dearmor -o /etc/apt/keyrings/postgresql.gpg \
    &&    echo    "deb    [signed-by=/etc/apt/keyrings/postgresql.gpg]
http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
    > /etc/apt/sources.list.d/pgdg.list \
 && apt-get update \
 && apt-get install -y postgresql-client-16 \
 && apt-get clean \
 && rm -rf /var/lib/apt/lists/*

RUN mkdir -p /etc/salt/minion.d

CMD ["salt-minion", "-l", "info"]
```

- Este archivo es un dockerfile. Es decir, un documento que define una build concreta de un container. Al final muchas veces no tenemos por qué ni siquiera interactuar con un ubuntu de un container si hemos realizado bien este paso. Este paso consiste en definir que tiene instalado y como lo tiene configurado la máquina antes de ni siquiera lanzar el compose up. Fijandonos en el código vemos comandos de instalación de paquetes, creación de directorios e interacciones con postgres.